

```
# 1. Upload dataset
from google.colab import files
files.upload()

# Example: cryptoprices.csv should have columns: Date, Close
cryptodataframe = pd.read_csv("cryptoprices.csv")

# Create a simple "up or down" label
cryptodataframe["Target"] = (cryptodataframe["Close"].
shift(-1) > cryptodataframe["Close"]).astype(int)
cryptodataframe = cryptodataframe.dropna()

X = cryptodataframe[["Close"]].values # using only 'Close'
for simplicity
y = cryptodataframe["Target"].values

# Normalize features
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.25, random_state=9)

# -----
# 2. Formation of Quantum circuit
# -----
n_qubits = 1
cdev = qmlib.cdevice("default.qubit", wires=n_qubits)

def myfeature_encoding(x):
    """Encode classical data into qubit rotation."""
    qmlib.RX(x[0], wires=0)

def myvariational_block(weights):
    """Simple variational layer."""
    qmlib.RY(weights[0], wires=0)
    qmlib.RZ(weights[1], wires=0)

@qmlib.qnode(cdev)
def myquantum_circuit(x, weights):
    feature_encoding(x)
    variational_block(weights)
    return qmlib.expval(qmlib.PauliZ(0))

# -----
# 3. Model + Training loop
# -----
def myquantum_model(x, weights):
    return myquantum_circuit(x, weights)

def cost(weights, X, y):
```

```
    pred = [myquantum_model(x, weights) for x in X]
    pred = np.sign(pred) # convert to {-1,1}
    y_mod = 2*y - 1      # convert labels {0,1} -> {-1,1}
    return np.mean(pred != y_mod)

# -----
# Initialize weights
# -----
myweights = np.random.randn(2, requires_grad=True)
myopt = qmlib.GradientDescentOptimizer(stepsize=0.1)

# -----
# Training
# -----
for epoch in range(15):
    myweights, _ = myopt.step_and_cost(lambda w: cost(w, X_
train, y_train), weights)
    train_loss = cost(weights, X_train, y_train)
    test_loss = cost(weights, X_test, y_test)
    print(f"Epoch {epoch+1:2d}: Train Error={train_loss:.3f},
Test Error={test_loss:.3f}")

# -----
# 4. Analytics and Inference
# -----
pred = [myquantum_model(x, myweights) for x in X_test]
pred_labels = (np.sign(pred) + 1)//2
print("\n Outcome Predictions:", pred_labels[:10])
print("Outcome True labels:      ", y_test[:10])
```

The output is:

```
Sample predictions: [0. 1. 1. 1. 1. 1. 1. 1. 1. 0.]
True labels:       [1 0 1 1 1 1 1 1 0 1]
```

In sample predictions, the output depicts either 1 or 0, where 1 predicts the price will go up and 0 predicts it will fall. 'True labels' gives the actual price increase and fall. This value is used to verify the accuracy of the model.

Student assessments using QML

The quantum machine learning code given below helps assess the knowledge of students in three different subjects. It uses quantum classifiers to predict whether they will pass or fail tests in these subjects.

```
from pennylane import numpy as np
import pennylane as qml
# QML Device Circuit Formation: 3 qubits for 3 subjects
mydev = qml.device("default.qubit", wires=3)
@qml.qnode(dev)
def studentcircuit(sscores, sweights):
```